ROS2 科研训练营

六周 • 从零到能上真机的计算图与闭环思维

你的名字 / 实验室

XXX 实验室

Version 0.1

2026 年 6 月 15 日

目录

第一章　上手准备：终端速览与环境验证 **1**

1.1　本周你将学会 1

1.2　终端不是黑魔法 2

1.3　ROS2 环境到底怎么才算好 2

1.4　workspace 与 colcon 的最小肌肉记忆 2

第二章　计算图思维：ROS2 的世界观 **6**

2.1　本周你将学会 6

2.2　为什么 ROS2 要把系统拆成图 7

2.3　topic、service、action 怎么选 7

第三章　空间计算图：TF2 与传感器接入 **11**

3.1　本周你将学会 11

3.2　TF2 为什么存在 11

3.3　map、odom、base_link 怎么分工 12

3.4　URDF 是给 TF 和可视化看的结构说明 13

第一章　上手准备：终端速览与环境验证

- 本周定位：在碰 ROS2 之前，先把终端和环境这两只小怪打掉。
- 本周成果：你能在 Ubuntu 终端里完成基本操作，并亲眼看到 ROS2 消息在流动。
- 本周边界：先本地原生安装，不碰 Docker；能跑通就收手。

1.1　本周你将学会

- 在终端里完成进目录、看文件、找关键字、看进程、停止命令等基本操作。
- 验证 Ubuntu 22.04 上的 ROS2 Humble 是否真的可用。
- 建立 `colcon build` 后要 `source install/setup.bash` 的肌肉记忆。
- 遇到报错时，知道该复制哪些上下文给 AI 帮你定位。

本周地图（30 秒读完）

你现在还没正式上车。本周只打两只小怪：第一，把终端用顺手；第二，把 ROS2 环境验证到能跑官方 demo。
建议分成两个 session：先用 30 分钟练终端，再用 60 分钟验证 ROS2 和 workspace。

先跑起来：5 分钟见现象

先用最小 demo 证明 ROS2 活着。打开两个终端，分别执行：

```
# terminal 1
source /opt/ros/humble/setup.bash
ros2 run demo_nodes_py talker

# terminal 2
source /opt/ros/humble/setup.bash
ros2 run demo_nodes_py listener
```

✔ listener 应该不断打印 `I heard` 开头的消息。你现在已经看见一个节点在发、另一个节点在收。
✘ 如果提示 `Package 'demo_nodes_py' not found`，先确认 ROS2 Humble 是否安装完整，再跑 `ros2 doctor` 看诊断。

1.2 终端不是黑魔法

终端只是一个更直接的操作界面。你输入命令，系统返回结果。刚开始不用懂很多，先记住“我在哪、这里有什么、我要进去哪里、我要看什么”。

任务	命令	你应该看到
我在哪	`pwd`	当前目录的完整路径
这里有什么	`ls`	当前目录下的文件和文件夹
进目录	`cd ~/ros2_ws`	提示符位置变化
建目录	`mkdir -p ~/ros2_ws/src`	新目录出现
看小文件	`cat package.xml`	文件内容直接打印出来
找关键字	`ros2 topic list | grep odom`	只显示含 odom 的行
中断命令	`Ctrl+C`	当前运行的程序停止
看环境变量	`echo $ROS_DISTRO`	正常应显示 humble

有两个省力按钮值得马上养成习惯。按 Tab 自动补全路径和命令，按方向键翻历史命令。你会少打很多字，也会少输错很多字。

1.3 ROS2 环境到底怎么才算好

安装命令本身很机械，适合让 AI 按你的系统版本生成。本教材更关心验证：你不是“好像装了”，而是能证明 ROS2 确实能跑。

先看发行版：

```
source /opt/ros/humble/setup.bash
echo $ROS_DISTRO
ros2 doctor
```

✔ `echo $ROS_DISTRO` 应该输出 `humble`。`ros2 doctor` 大部分检查应通过，个别 warning 可以先记下来，不要在第 0 周钻太深。

✘ 如果 `ros2 doctor` 报一串 error，把完整输出、你的 Ubuntu 版本和你执行过的安装命令一起交给 AI 排查。别只说“它坏了”，AI 也怕没上下文。

计算图视角

第 0 周只预告一眼计算图：talker/listener 其实已经是一个最小 ROS2 系统。
talker 是发布节点，listener 是订阅节点，中间那条数据流叫 topic。第 1 周我们会正式拆开它，讲清楚为什么 ROS2 要这样组织机器人系统。

1.4 workspace 与 colcon 的最小肌肉记忆

ROS2 写代码通常放在一个 workspace 里。你可以把 workspace 理解成“我的 ROS2 项目工位”，`src` 里放源码，build 后生成可运行的结果。

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws
colcon build
source install/setup.bash
```

最容易踩的坑是忘了 source。build 只是把包编出来，source 才是告诉当前终端：“去这个 workspace 里找我刚编好的包。”每开一个新终端，都要重新 source。

动手里程碑

1. 练一组终端基本动作。

```
mkdir -p ~/ros2_bootcamp_scratch
cd ~/ros2_bootcamp_scratch
pwd
echo "ros2 humble is ready" > note.txt
cat note.txt
grep humble note.txt
```

✔ 你应该看到当前目录路径、文件内容，以及含 `humble` 的那一行。

2. 验证 ROS2 自检。

```
source /opt/ros/humble/setup.bash
ros2 doctor
```

✔ 你应该看到诊断结果基本通过。
✘ 如果报 locale 或 GPG key 相关错误，优先把完整错误粘给 AI，不要凭感觉乱改系统文件。

3. 跑通 talker/listener。

```
ros2 run demo_nodes_py talker
# open another terminal
ros2 run demo_nodes_py listener
```

✔ listener 持续打印收到的消息，就算通过。

样例代码导读

本周基本不依赖样例库。你只需要建好自己的 `~/ros2_ws`，并确认能 build、能 source、能运行 ROS2 官方 demo。
样例库里的 `docker/` 是后面部署与复现时的可选进阶。初学阶段先别上来就用 Docker，它会把“我不会 ROS2”和“我不会容器”两个问题叠在一起。

AI 工具箱

安装环境这类机械步骤适合交给 AI 生成，但你要明确限定系统和目标。

```
帮我在 Ubuntu 22.04 上原生 apt 安装 ROS2 Humble：给可以直接复制执行的命令，
每一步用一句话说明在干嘛；装完再给一份怎么验证环境 OK 的检查，
至少包含 ros2 doctor，以及开两个终端跑 talker / listener 看一发一收。
我是新手，命令请按顺序、别跳步，不要使用 Docker。
```

验证 AI 给你的东西

1. 能不能跑：执行 `ros2 doctor`，确认没有成片 error。
2. demo 是否通：两个终端分别跑 talker 和 listener，listener 能持续收到消息。
3. 环境变量是否对：`echo $ROS_DISTRO` 应输出 `humble`。
4. workspace 是否对：在 `~/ros2_ws` 下执行 `colcon build` 后，能成功 `source install/setup.bash`。
5. 出错了怎么办：把完整报错、Ubuntu 版本、执行过的命令一起贴给 AI。

AI 工具箱

报错时用这条提示词，不要只丢一句“报错了”。

```
我在 Ubuntu 22.04 原生安装 ROS2 Humble 时遇到这个错误：[粘贴完整报错]。
这是我刚才执行过的命令：[粘贴命令历史]。
请帮我判断最可能的原因，并给出一步步修复命令。每一步都说明为什么要做。
```

高频排错

- 现象：`ros2` 命令找不到。先查有没有执行 `source /opt/ros/humble/setup.bash`。
- 现象：中文或 locale 报错。检查系统 locale 是否是 UTF-8。
- 现象：apt 装包失败。常见原因是源、GPG key 或网络代理问题。
- 现象：新终端里包又找不到。每个新终端都要 source ROS2 和你的 workspace。
- 现象：不知道该不该加 sudo。装系统包通常需要，改自己 workspace 里的文件通常不需要。

本周检查点（过了才算掌握）

- ☐ 我能在终端里建目录、进目录、看文件、grep 关键字。
- ☐ 我能跑通 `ros2 doctor`，并知道 warning 和 error 的区别。
- ☐ 我能用两个终端跑 talker/listener，并说出谁在发、谁在收。

- □ 我知道 build 后为什么要 source。

建议时长 & 延伸阅读

建议时长：1–2 小时。
这周的成功标准是环境不再吓人。你已经能让 ROS2 发出第一条消息了，下一周我们把这条消息放进一张真正的计算图里。

第二章　计算图思维：ROS2 的世界观

- 本周定位：建立“一切皆图”的世界观。
- 本周成果：看见一个 ROS2 系统的计算图，并写出第一个最小节点。
- 本周节奏：先看现象，再拆 node、topic、service、action、parameter。

2.1　本周你将学会

- 说清 **node**、**topic**、**service**、**action**、**parameter** 各自解决什么问题。
- 用 `ros2` 命令行和 `rqt_graph` 把一个跑起来的系统“看”成计算图。
- 跑通地面机器人样例，并解释图里每个节点在做什么。
- 写出一个最小 publisher 与 subscriber，并让它出现在图里。

本周地图（30 秒读完）

你现在站在 ROS2 的入口处。本周只做三件事：先跑一个能看见的图，再学会把系统拆成节点和数据流，最后亲手往图里塞一个自己的小节点。
建议分成三个 session：第一段看图，第二段拆概念，第三段写最小节点。

先跑起来：5 分钟见现象

先别急着理解所有术语。打开已经配置好 ROS2 环境的终端，跑一个最小的 talker/listener，再打开图。

```
ros2 run demo_nodes_py talker
# open another terminal
ros2 run demo_nodes_py listener
# open a third terminal
rqt_graph
```

✔ 你应该看到 listener 不断打印收到的字符串，`rqt_graph` 里出现一个发布节点、一个订阅节点和中间的话题。

✘ 如果提示 `ros2: command not found`，先回到第 0 周检查有没有执行 `source /opt/ros/humble/setup.bash`。

2.2 为什么 ROS2 要把系统拆成图

机器人系统复杂，不是因为每个函数都难，而是因为东西太多：传感器在发数据，控制器在发指令，规划器在等目标，硬件驱动在和真实世界打交道。ROS2 的做法是把这些职责拆成一块块可以独立运行、独立替换的小部件。

node 就是“负责一件事的小进程”。一个相机驱动是 node，一个键盘控制器是 node，一个滤波器也是 node。拆成 node 的好处不是好看，而是出问题时你能问：“是哪一个职责坏了？”

topic 是持续流动的数据管道。图像、雷达、里程计这类源源不断的数据适合走 topic，因为发送者不需要等接收者点头，接收者也可以有很多个。

service 是一次性请求和回答。比如“请现在保存一张地图”“请查询这个参数”，你要的是一个明确结果。

action 是耗时任务。比如“导航到门口”不是一问一答，它需要进度反馈，也可能被取消。

parameter 是运行时可调的旋钮。速度上限、话题名、滤波阈值这类东西不应该写死在代码里。

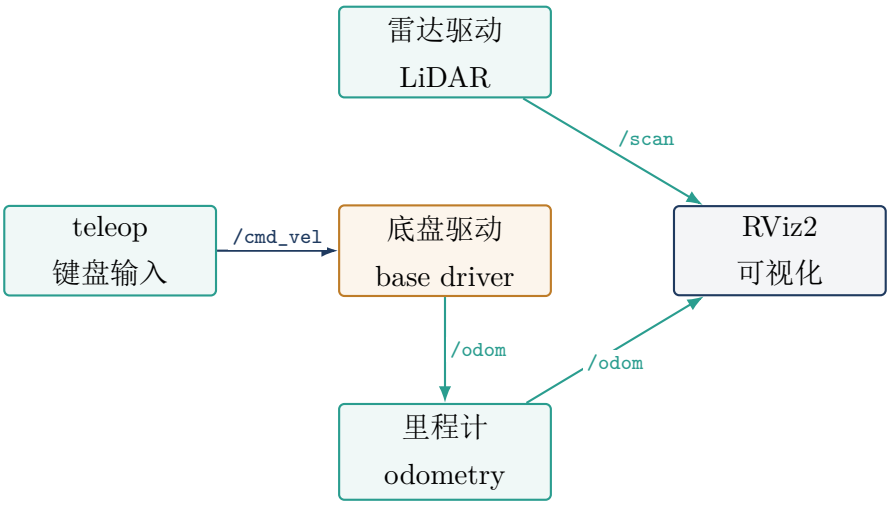


图 2.1: 一个最小地面机器人系统的计算图草图。

看图时先不要背术语，先问三句话：谁在生产数据，谁在消费数据，中间走的是什么名字的数据流。

2.3 topic、service、action 怎么选

一个实用判断法：如果数据像水龙头一样一直流，用 topic；如果像按门铃一样问一次答一次，用 service；如果像点外卖一样要等一段时间、想看进度、还可能取消，用 action。

机制	适合什么	典型例子
topic	持续数据流，多对多，异步	/scan、/image、/odom
service	一次请求，一次回答	保存地图、查询状态、重置计数器
action	长任务，有反馈，可取消	导航到目标点、执行轨迹、抓取任务
parameter	运行时配置	速度上限、frame 名、阈值

QoS 暂时只记一个拇指法则：传感器数据常用 best effort，因为新数据比旧数据重要；控制指令常用 reliable，因为丢一条指令可能让系统行为变怪。后面遇到无线网络和真机部署时，我们再认真处理它。

计算图视角

本周的主题就是计算图本身。把一个 ROS2 系统画出来时，node 是方块，topic/service/action 是方块之间的边，parameter 是挂在 node 身上的旋钮。

调试时先看图：`ros2 node list` 看有哪些方块，`ros2 topic list` 看有哪些数据流，`rqt_graph` 看它们有没有连起来。

闭环视角

先埋一颗种子：`/cmd_vel` 是“我想让机器人怎么动”，`/odom` 是“机器人实际动成什么样”。这两条 topic 之间藏着一个闭环。

第 3 周我们会把这个环正式拆开：感知、估计、控制、执行，再回到感知。

动手里程碑

1. 跑通地面机器人 bringup。

```
ros2 launch ground_robot bringup.launch.py
```

✔ 你应该看到 RViz2 里出现机器人模型和雷达点云。

✘ 如果 launch 文件找不到，先确认样例库已经放在当前 workspace，并且 build 后执行过 `source install/setup.bash`。

2. 让机器人动起来，并看里程计变化。

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
ros2 topic echo /odom
```

✔ 你按方向键时，`/odom` 里的位姿或速度数值应该变化。

✘ 如果 `/cmd_vel` 有输出但车不动，优先检查底盘驱动节点是否订阅了同一个话题名。

3. 写一个最小节点，把字符串从一个 topic 转发到另一个 topic。

```
ros2 pkg create --build-type ament_python week1_echo
colcon build --packages-select week1_echo
```

```
source install/setup.bash
```

✔ 你的节点运行后，应能在 rqt_graph 里看到自己新加的方块。

样例代码导读

本周关联样例代码：

- ground_robot/bringup/：负责一键拉起仿真、RViz2、robot state publisher 等节点。
- ground_robot/teleop_control/：展示 /cmd_vel 如何进入底盘控制链路。
- exercises/week1/：放最小 publisher/subscriber 练习，包含 TODO 版和参考答案。

读代码时只抓三件事：这个文件启动了哪些 node，它们发布或订阅哪些 topic，哪些参数决定了它们的名字和行为。

AI 工具箱

当你要写第一个节点时，骨架可以交给 AI。你的任务是看懂它在图里的位置，并把处理逻辑填进去。

```
为下面这个数据流写一个 ROS2 Humble 的 Python 处理节点骨架：
订阅 [输入话题/消息类型]，对数据做 [处理描述]，发布到 [输出话题/消息类型]。
要求：声明可调参数 [参数列表]，设置合适的 QoS，加 get_logger 日志，
用面向对象写法继承 Node。给出完整 Python 文件，以及 setup.py 里要加的 entry_point。
```

验证 AI 给你的东西

1. 能不能跑：执行 ros2 run <package> <entry_point>，节点启动且不报 import 错误。
2. 在图里对不对：用 rqt_graph 看新节点是否连在预期的输入和输出 topic 之间。
3. 话题类型对不对：用 ros2 topic info <topic> 检查发布端和订阅端的 message type 是否一致。
4. 参数是否生效：运行时传一个不同参数值，确认日志或输出行为真的改变。
5. 出错了怎么办：把完整报错、节点代码、ros2 topic list 和 ros2 node list 一起丢给 AI 排查。

AI 工具箱

如果 topic、service、action 还是分不清，用这条提示词换一条理解路径。

```
我在学 ROS2，是机器人初学者。请用一个生活化的比喻，外加一张 ASCII 流程图，
解释 topic、service、action 三者的区别，以及在什么场景该用哪个。
不要堆术语，假设我什么都不懂。
```

高频排错

- 现象：`ros2` 命令找不到。原因通常是没有 source ROS2 环境。查 `echo $AMENT_PREFIX_PATH`。
- 现象：build 后新包找不到。原因通常是没有 source workspace 的 `install/setup.bash`。
- 现象：`rqt_graph` 里看不到别人的节点。检查 `ROS_DOMAIN_ID` 是否一致。
- 现象：仿真里时间相关报错。检查节点是否需要 `use_sim_time`。

本周检查点（过了才算掌握）

- ☐ 我能画出 talker/listener 的计算图，并指出 node 和 topic。
- ☐ 我能解释为什么 `/scan` 是 topic，而不是 service。
- ☐ 我能用 `ros2 node list`、`ros2 topic list`、`rqt_graph` 查一个系统。
- ☐ 我写的最小节点能出现在图里，并且输入输出 topic 正确。

建议时长 & 延伸阅读

建议时长：6–8 小时。
你现在已经能把一个 ROS2 系统看成图，并知道调试时先去找哪些方块和连线。下一周我们给这张图加上空间：同一份数据到底是在哪个 frame 里说话。

第三章　空间计算图：TF2 与传感器接入

- 本周定位：给计算图加上空间维度。
- 本周成果：看懂一棵 TF 树，把传感器数据接进图，并让消息带上正确 frame。
- 本周边界：URDF 只讲作用和验证，不教手写 XML。

3.1　本周你将学会

- 解释为什么坐标变换需要 TF2 统一管理。
- 看懂 `map -> odom -> base_link -> sensor` 这类 TF 树，并知道每段由谁发布。
- 把相机、雷达、IMU 这类传感器理解成计算图里的节点。
- 用 AI 生成简化 URDF/xacro，并用工具验证它是否真的对。
- 写一个最小数据处理节点，把 `/scan` 处理后发布成 `/scan_filtered`。

本周地图（30 秒读完）

第 1 周你已经会看“谁和谁连着”。第 2 周要多问一句：这份数据是在谁的坐标系里说话？建议分成三个 session：先看 TF 树，再接传感器 frame，最后插入一个处理节点。

先跑起来：5 分钟见现象

先在 RViz2 里把 TF 看出来。启动样例后，打开 TF 显示，并导出一份 frames 图。

```
ros2 launch ground_robot bringup.launch.py
# open another terminal
ros2 run tf2_tools view_frames
```

✔ 你应该得到一个 `frames.pdf`，里面能看到 `map`、`odom`、`base_link` 和各传感器 frame 的连接关系。

✘ 如果提示某个 frame 不存在，不要先猜。先用 `ros2 topic echo /tf_static` 和 `ros2 topic echo /tf` 看有没有对应发布者。

3.2　TF2 为什么存在

机器人世界里没有“绝对的前后左右”，只有“相对于谁的前后左右”。雷达点在雷达坐标系里，相机图像在相机坐标系里，机器人底盘在 `base_link` 里，地图在 `map` 里。

如果每个节点都自己算坐标变换，系统很快会乱：有人用旧时间戳，有人忘了传感器安装偏移，有人把父子 frame 写反。TF2 的价值就是把这些变换集中管理，并且带着时间戳查询。

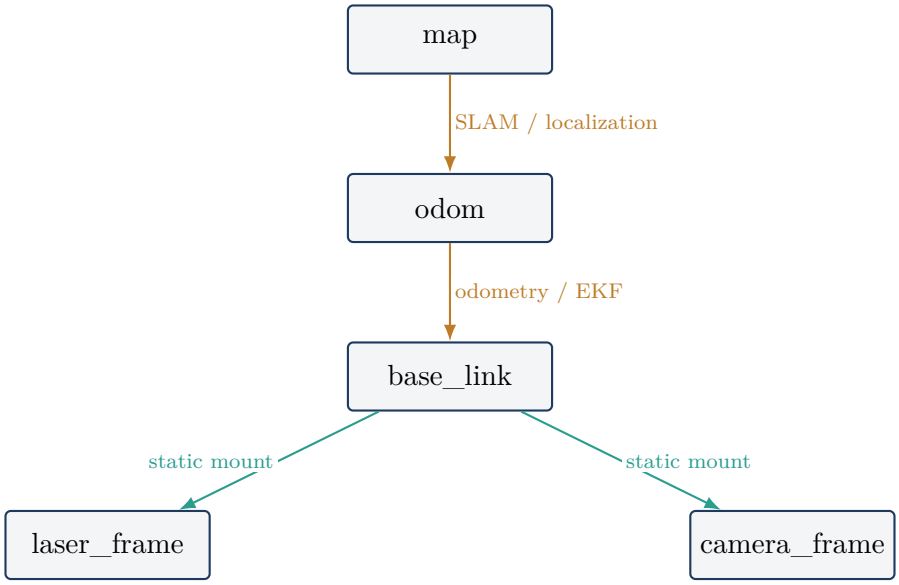


图 3.1: 常见移动机器人 TF 树：动态变换负责运动，静态变换负责安装关系。

3.3 map、odom、base_link 怎么分工

`base_link` 是机器人本体基准。你可以把它理解成贴在底盘中心的一张小坐标纸。

`odom -> base_link` 是连续的短期运动估计。它平滑、不跳变，适合控制器用；但它会慢慢漂移。

`map -> odom` 是全局修正。SLAM 或定位节点发现“我其实在地图这里”时，会调整这段变换，把漂移纠回来。它不适合直接拿来做底层控制，因为可能有离散修正。

传感器 frame 通常是静态的。雷达装在车头，摄像头装在支架上，这些相对 `base_link` 的位置不会随着车移动而变化。

计算图视角

TF 也是计算图的一部分。普通数据走 `/scan`、`/image`、`/imu`，坐标关系走 `/tf` 和 `/tf_static`。传感器节点发布数据时，message header 里的 `frame_id` 告诉系统“这份数据属于哪个 frame”。RViz2 能把点云画到正确位置，是因为它同时拿到了数据和 TF 树。

闭环视角

`odom -> base_link` 是上周埋下的闭环线索：它是“执行之后，机器人实际去哪了”的估计结果。

下周我们会把这段变换背后的状态估计拆开：轮式里程计、IMU、EKF，以及为什么只有感知反馈回来，控制才算真的闭环。

3.4 URDF 是给 TF 和可视化看的结构说明

URDF 描述机器人有哪些 link，link 之间通过哪些 joint 连起来，每个 joint 的 parent、child 和 origin 是什么。xacro 是带参数和宏的 URDF，适合把重复结构写得短一点。

本课不要求你手写 URDF 的 XML。你要会读四个关键词：`link` 是部件，`joint` 是连接，`parent` 和 `child` 决定树方向，`origin` 决定相对位置和姿态。

`robot_state_publisher` 会读取 URDF，再结合 `joint_states`，发布机器人各 link 之间的 TF。也就是说，URDF 不是摆设，它会真的进入计算图。

动手里程碑

1. 查看 TF 树。

```
ros2 launch ground_robot bringup.launch.py
ros2 run tf2_tools view_frames
```

✔ 打开 `frames.pdf`，确认 `base_link` 到传感器 frame 的边存在。

2. 监听静态 TF。

```
ros2 topic echo /tf_static
```

✔ 你应该能看到传感器相对底盘的固定安装关系。

✘ 如果没有输出，检查是否启动了 `robot_state_publisher` 或静态变换发布器。

3. 插入一个点云或雷达处理节点。

```
ros2 run ground_robot_perception scan_filter_node \
  --ros-args -p input_topic:=/scan -p output_topic:=/scan_filtered
```

✔ `ros2 topic list` 应出现 `/scan_filtered`，RViz2 里可以对比过滤前后的数据。

样例代码导读

本周关联样例代码：

- `ground_robot/description/`：机器人 URDF/xacro、传感器安装 frame、可视化配置。
- `ground_robot/perception/`：订阅、处理、发布传感器数据的节点示范。
- `exercises/week2/`：添加一个传感器 frame，并写 `/scan` 到 `/scan_filtered` 的练习。

读这些文件时先不追 XML 细节，先确认三件事：frame 名是什么，父子关系是什么，这个 frame 对应的数据话题是什么。

AI 工具箱

URDF/xacro 属于机械样板，生成交给 AI。你负责给清楚需求，并验证它生成的结构是否进入 TF 树。

```
为一个差速驱动地面机器人生成 ROS2 Humble 的 URDF/xacro:
包含底盘、两个驱动轮、一个万向轮、顶部一个 2D LiDAR、前向一个相机。
给每个 link 合理的惯性与碰撞体，传感器给正确的 joint 和 origin。
输出完整可用的 xacro，并在每一段上方用注释说明这段在描述什么、关键字段含义。
```

验证 AI 给你的东西

1. 能不能跑：用 `robot_state_publisher` 加载 xacro，节点启动且不报 XML 错。
2. 在图里对不对：运行 `ros2 run tf2_tools view_frames`，确认新传感器 frame 出现在树上。
3. 关键字段对不对：检查每个 `joint` 的 `parent`、`child`、`origin` 是否符合物理安装。
4. 可视化对不对：在 RViz2 里看模型，传感器不要飘在奇怪位置，也不要埋进底盘。
5. 出错了怎么办：把 xacro、报错和 `frames.pdf` 结果一起丢给 AI，用 TF 调试提示词。

AI 工具箱

TF 报错时，用具体工具输出喂给 AI。

```
我的 ROS2 Humble 系统报这个 TF 错误：[完整报错]。
这是我的 TF 树 view_frames 结果：[描述或粘贴关键内容]，
这是相关节点 ros2 node list：[粘贴]。
请逐步帮我定位是哪段 TF 没发布、父子 frame 写反，还是时间戳有问题，
并给出最可能的修复步骤。
```

AI 工具箱

如果 map、odom、base_link 还是混，用这条提示词换个说法。

```
解释 TF2 里 map、odom、base_link 这三个坐标系：为什么要分三层？
odom->base_link 和 map->odom 分别由谁发布、各有什么特性（连续/离散、漂移/不漂移）？
用一个“你在商场里找路”的比喻来讲，不要堆公式。
```

高频排错

- 现象：`frame does not exist`。先查这个 frame 是否真的被发布，再查名字是否拼错。
- 现象：TF 外推失败。常见原因是消息时间戳和 TF 时间戳不在同一时间线上。
- 现象：RViz2 里传感器位置明显不对。检查 URDF 的 `origin` 和 parent/child 是否反

了。

- 现象：`/tf_static` 没有静态安装关系。检查 `robot_state_publisher` 是否加载了正确 URDF。
- 现象：处理后的点云不显示。检查输出消息的 `header.frame_id` 有没有保留或正确设置。

本周检查点（过了才算掌握）

- ☐ 我能画出样例机器人从 `map` 到传感器 frame 的 TF 树。
- ☐ 我能说出 `map -> odom` 和 `odom -> base_link` 的区别。
- ☐ 我能用 AI 生成一个简化 URDF，并用 TF 工具验证它。
- ☐ 我能插入一个传感器处理节点，让输出 topic 出现在图里。
- ☐ 我知道 TF 报错时先查 frame 名、发布者和时间戳。

建议时长 & 延伸阅读

建议时长：8–10 小时。
你现在能把“数据流”放进“空间关系”里看了。下一周我们把 `odom -> base_link` 背后的估计和控制讲透，正式进入闭环思想。